EA2. Creación de repositorio
Procesamiento funcional de datos para un sistema de gestión de transporte urbano (TecnoMovil Data)

Ana María Osorio Tabarez

Grupo PREICA2602B020046

Institución Universitaria Digital de Antioquia (IU Digital)

Tecnología en Desarrollo de Software

Programación Orientada a Objetos II

Docente: Jorge Armando Julio

5 de noviembre de 2026

Introducción

La Alcaldía de TecnoValle adelanta el proyecto TecnoMovil Data para modernizar su sistema de transporte urbano, el cual moviliza más de 450.000 pasajeros por día y genera entre 6 y 12 millones de registros diarios provenientes de validadores, sensores, aplicaciones móviles y dispositivos IoT (Institución Universitaria Digital de Antioquia [IU Digital], 2026). La plataforma actual procesa esa información con un enfoque imperativo y mutable, lo que produce tiempos de procesamiento de hasta dos horas, errores de conteo por condiciones de carrera y un alto costo de mantenimiento.

Este informe documenta la solución de la evidencia de aprendizaje EA2: un módulo escrito en Java, sin frameworks externos, que reemplaza el procesamiento secuencial y mutable por un diseño declarativo basado en funciones puras, lambdas, colecciones inmutables y pipelines de Streams. El módulo resuelve seis análisis sobre un conjunto de datos simulado de más de un millón de registros: afluencia por estación, horas pico, rutas más utilizadas, patrones de viaje por usuario, tiempo promedio entre estaciones y detección de sobrecarga en rutas.

El documento presenta los objetivos, el desarrollo (análisis y arquitectura, conceptos funcionales aplicados, implementación de cada requerimiento, pruebas y evidencias de ejecución), las conclusiones, el enlace al repositorio de GitHub y, como anexo, el código fuente completo.

Objetivos

Objetivo general

Aplicar los conceptos de la programación funcional en Java para diseñar un módulo declarativo y escalable de procesamiento de datos del sistema de transporte urbano TecnoMovil Data.

Objetivos específicos

- Modelar los registros y los resultados del análisis con estructuras inmutables que impidan modificaciones accidentales de los datos originales.
- Implementar los seis análisis solicitados mediante funciones puras, lambdas, funciones de orden superior y pipelines de Streams, evitando efectos secundarios.
- Verificar la corrección del módulo con pruebas sobre un conjunto verificable a mano y comprobar que la ejecución paralela produce los mismos resultados que la secuencial.
- Documentar la solución con evidencias de ejecución y alojarla en un repositorio de GitHub en la rama principal.

Desarrollo

1. Análisis del caso y arquitectura de la solución

El módulo se diseñó siguiendo el patrón “núcleo funcional, borde imperativo”. Toda la lógica de análisis vive en la clase AnalizadorTransporte, cuyos métodos son puros: reciben datos, devuelven resultados nuevos y no alteran ningún estado externo. Las únicas operaciones con efectos secundarios (imprimir en consola) están aisladas en la clase Main. La Tabla 1 resume la organización del proyecto.

Tabla 1

Organización del proyecto Java (paquete co.edu.iudigital.tecnomovil)

Paquete / clase	Responsabilidad
modelo.Accion	Enum con las acciones ENTRADA y SALIDA (tipo cerrado, inmutable).
modelo.Registro	Record inmutable con idUsuario, ruta, estacion, accion y timestamp (sección 3.1 del caso).
modelo.Ruta / TramoViaje / EstadoRuta	Records auxiliares: definición de ruta, tramo entre estaciones y resultado de sobrecarga.
datos.Catalogo	Lista inmutable de 8 rutas (T1–T4 troncales, A1–A4 alimentadoras), 18 estaciones y capacidad horaria simulada.
datos.GeneradorDatos	Simulador determinista de registros (semilla fija); cada viaje es función pura de (semilla, usuario, viaje).
funcional.Funciones	Lambdas con nombre: predicados, clasificadores y composiciones reutilizables.
funcional.AnalizadorTransporte	Núcleo funcional: los seis análisis a)–f) con Streams.
Main	Capa imperativa: genera datos, invoca el análisis e imprime resultados.
PruebasFuncionales	Pruebas sin frameworks sobre datos pequeños verificables a mano.

Nota. Elaboración propia.

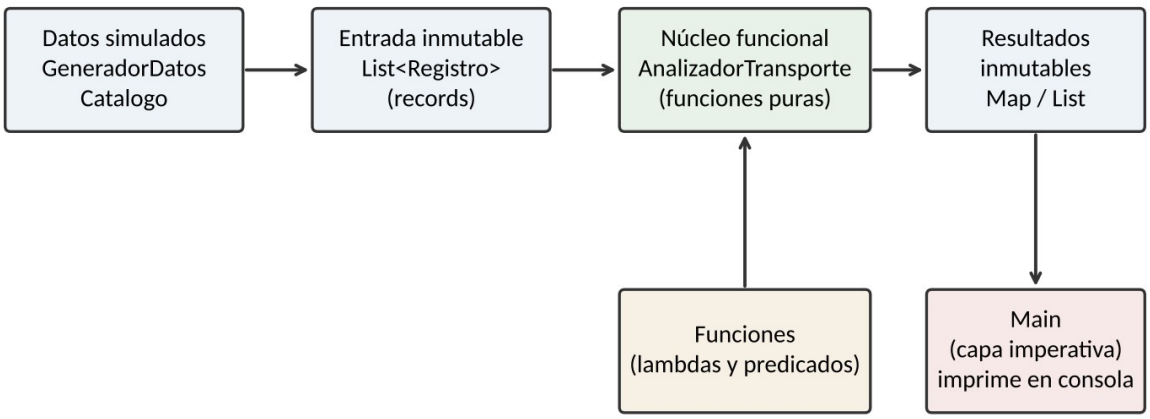
Para simular el volumen del caso se generan 200.000 usuarios con entre dos y cuatro viajes cada uno, lo que produce 1.198.944 registros de entrada y salida para el 4 de noviembre de 2026. Los viajes se concentran en las horas pico de la mañana (7:00–8:59) y de la tarde (17:00–18:59) mediante una lista ponderada de horas, y cada ruta tiene un peso de popularidad distinto. El generador es determinista

(semilla 2026), por lo que cualquier integrante obtiene exactamente los mismos resultados. El tamaño puede aumentarse hasta los 6–12 millones de registros del caso modificando la constante USUARIOS y la memoria de la JVM. Como la última estación de cada ruta solo recibe salidas, la estación Estadio no aparece en el conteo de ingresos.

La Figura 1 presenta el flujo de datos entre las clases: los datos simulados se convierten en una lista inmutable de registros, el núcleo funcional produce resultados inmutables y solo Main imprime en consola. La Figura 2 resume la forma general de los pipelines de Streams usados en los seis análisis.

Figura 1

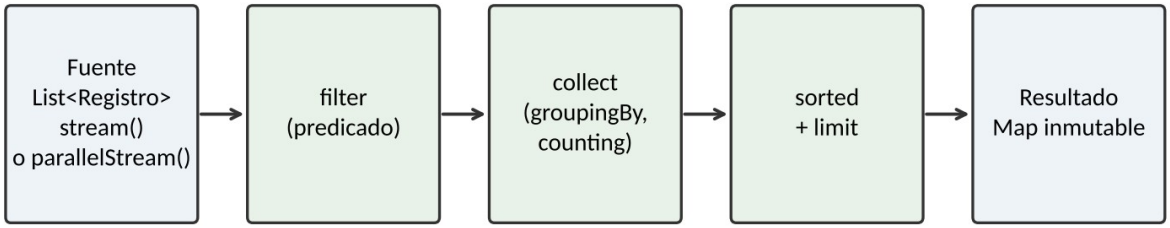
Arquitectura del módulo funcional



Nota. Elaboración propia. Las flechas indican el sentido del flujo de datos.

Figura 2

Esquema general de un pipeline de Streams



Nota. Elaboración propia. Cada análisis varía el predicado, el clasificador y el criterio de orden.

2. Conceptos de programación funcional aplicados

2.1 Funciones puras

Una función pura siempre devuelve el mismo resultado para los mismos argumentos y no produce efectos secundarios. En el proyecto, TramoViaje.entre decide si dos registros consecutivos forman un tramo válido usando únicamente sus parámetros, y estadoDeRuta calcula la ocupación de una ruta sin consultar ni modificar variables externas. Los predicados de Funciones (por ejemplo ES_ENTRADA) también son puros. Gracias a esto el resultado no depende del orden de ejecución ni del número de hilos, lo que elimina las condiciones de carrera del sistema anterior (Bloch, 2018, ítem 46).

```
public static Optional<TramoViaje> entre(Registro a, Registro b) {
   boolean valido = a.ruta().equals(b.ruta())
         && a.accion() == Accion.ENTRADA
         && b.accion() == Accion.SALIDA
         && !a.estacion().equals(b.estacion());
   return valido
         ? Optional.of(new TramoViaje(a.ruta(), a.estacion(), b.estacion(),
         Duration.between(a.timestamp(), b.timestamp()).getSeconds()))
         : Optional.empty();
}
```

Nota. Función pura: el resultado solo depende de a y b.

2.2 Inmutabilidad

Ningún método modifica las listas originales. Los registros y demás modelos son record (campos finales, sin setters; Oracle, s. f.-a). El analizador guarda una copia defensiva con List.copyOf, el generador

entrega la lista con Stream.toList() y todos los resultados se devuelven envueltos en Collections.unmodifiableMap o como List.copyOf. La clase Main comprueba la propiedad intentando agregar un elemento a la lista original, lo que lanza UnsupportedOperationException (Figura 4). Esta práctica sigue la recomendación de minimizar la mutabilidad (Bloch, 2018, ítem 17).

```
public record Registro(String idUsuario,
      String ruta,
      String estacion,
      Accion accion,
      LocalDateTime timestamp) {

   public Registro {
      Objects.requireNonNull(idUsuario, "idUsuario");
      Objects.requireNonNull(ruta, "ruta");
      Objects.requireNonNull(estacion, "estacion");
      Objects.requireNonNull(accion, "accion");
      Objects.requireNonNull(timestamp, "timestamp");
   }
}
```

Nota. Record inmutable con validación en el constructor compacto.

2.3 Lambdas y composición

Las lambdas permiten expresar el comportamiento como valor. En Funciones se definen predicados y clasificadores con nombre, y se combinan con Predicate.and y Function.andThen: ES_ENTRADA.and(deRuta("T1")) filtra los ingresos de una ruta, y FRANJA_HORARIA compone HORA_DEL_DIA con una lambda de formato para producir etiquetas como “07:00-07:59” (Oracle, s. f.-b).

```
// ---- Predicados ----
public static final Predicate<Registro> ES_ENTRADA = r -> r.accion() == Accion.ENTRADA;
public static final Predicate<Registro> ES_SALIDA = r -> r.accion() == Accion.SALIDA;

/** Fábrica de predicados: función de orden superior que RETORNA una función. */
public static Predicate<Registro> deRuta(String ruta) {
   return r -> r.ruta().equals(ruta);
}

// ---- Clasificadores (funciones de extracción) ----
public static final Function<Registro, String> ESTACION = Registro::estacion;
public static final Function<Registro, String> RUTA = Registro::ruta;
public static final Function<Registro, Integer> HORA_DEL_DIA = r -> r.timestamp().getHour();

// ---- Composición de funciones (andThen) ----
public static final Function<Registro, String> FRANJA_HORARIA =
      HORA_DEL_DIA.andThen(h -> String.format("%02d:00-%02d:59", h, h));
```

Nota. Lambdas con nombre, fábrica de predicados y composición con andThen.

2.4 Funciones de orden superior

Son funciones que reciben o retornan otras funciones. El método privado contarPor(filtro, clasificador) recibe un Predicate y una Function y es reutilizado por los análisis a), b) y c), evitando repetir el mismo pipeline. deRuta retorna un predicado, ordenar recibe un Comparator y detectarSobrecarga recibe la capacidad horaria como una ToIntFunction, de modo que el criterio de capacidad se inyecta desde fuera.

```
private <K> Map<K, Long> contarPor(Predicate<Registro> filtro, Function<Registro, K> clasificador) {
   return flujo(registros)
         .filter(filtro)
         .collect(Collectors.groupingBy(clasificador, Collectors.counting()));
}
```

Nota. Función de orden superior reutilizada por tres requerimientos.

2.5 Streams y paralelización

Todos los análisis son pipelines declarativos: una fuente (la lista de registros), operaciones intermedias (filter, sorted, flatMap, map) y una operación terminal (collect) con colectores como groupingBy, counting, mapping y averagingLong (Oracle, s. f.-c). Los pipelines no usan variables compartidas ni forEach para acumular resultados, requisito para que sean seguros en paralelo. El

método privado flujo selecciona stream() o parallelStream() según el parámetro paralelo del constructor. La Tabla 2 relaciona cada requerimiento con su pipeline.

Tabla 2

Pipelines de Streams por requerimiento

Req.	Método	Operaciones principales
a	afluenciaPorEstacion	filter(ES_ENTRADA) -> groupingBy(estación, counting) -> sorted -> limit -> LinkedHashMap inmutable
b	horasPico / horasValle	filter(ES_ENTRADA) -> groupingBy(hora, counting) -> sorted descendente o ascendente
c	rutasMasUtilizadas	filter(ES_ENTRADA) -> groupingBy(ruta, counting) -> sorted -> limit(n)
d	patronesDeViaje	sorted(timestamp) -> groupingBy(idUsuario, mapping(estación, toList))
e	tiempoPromedioEntreEstaciones	groupingBy(usuario) -> flatMap(tramosDeUsuario) -> groupingBy(tramo, averagingLong)
f	detectarSobrecarga	filter(ES_ENTRADA) -> groupingBy(ruta, groupingBy(hora, counting)) -> map(estadoDeRuta) -> sorted -> toList

Nota. El símbolo -> indica el orden de las operaciones del pipeline.

3. Implementación de los requerimientos

3.1 Afluencia por estación (a)

Se filtran los registros de entrada, se agrupan por estación y se cuenta. El resultado se ordena de mayor a menor afluencia, con desempate alfabético para que la salida sea determinista.

```
public Map<String, Long> afluenciaPorEstacion() {
   return ordenarDescendente(contarPor(ES_ENTRADA, ESTACION), Integer.MAX_VALUE);
}
```

3.2 Horas pico (b)

Se agrupan los ingresos por hora del día (HORA_DEL_DIA) y se ordenan. horasPico usa orden descendente y horasValle ascendente; ambos reutilizan contarPor y el ordenamiento genérico.

```
public Map<Integer, Long> horasPico(int n) {
   return ordenarDescendente(contarPor(ES_ENTRADA, HORA_DEL_DIA), n);
}

public Map<Integer, Long> horasValle(int n) {
   Map<Integer, Long> porHora = contarPor(ES_ENTRADA, HORA_DEL_DIA);
   Comparator<Map.Entry<Integer, Long>> ascendente =
         Map.Entry.<Integer, Long>comparingByValue().thenComparing(Map.Entry.comparingByKey());
   return ordenar(porHora, ascendente, n);
}
```

3.3 Rutas más utilizadas (c)

Mismo patrón con la ruta como clasificador; limit(n) conserva las n rutas con más ingresos.

```
public Map<String, Long> rutasMasUtilizadas(int n) {
   return ordenarDescendente(contarPor(ES_ENTRADA, RUTA), n);
}
```

3.4 Patrones de viaje por usuario (d)

Los registros se ordenan por timestamp (ordenamiento estable) y se agrupan por usuario; mapping transforma cada registro en el nombre de su estación y toList conserva el orden de aparición. El TreeMap mantiene los usuarios ordenados y el resultado final es inmutable.

```
public Map<String, List<String>> patronesDeViaje() {
   return flujo(registros)
         .sorted(Comparator.comparing(Registro::timestamp))
         .collect(Collectors.collectingAndThen(
         Collectors.groupingBy(Registro::idUsuario, TreeMap::new,
         Collectors.mapping(ESTACION,
         Collectors.collectingAndThen(Collectors.toList(), List::copyOf))),
         Collections::unmodifiableMap));
}
```

3.5 Tiempo promedio entre estaciones (e)

Se agrupan los registros por usuario. Para cada usuario, tramosDeUsuario ordena sus registros por tiempo y forma pares consecutivos con IntStream.range; TramoViaje.entre conserva solo los pares

ENTRADA -> SALIDA de la misma ruta y estaciones distintas. Finalmente se agrupan los tramos por ruta-origen-destino y se promedian los segundos, que se convierten a minutos.

```
public Map<String, Double> tiempoPromedioEntreEstaciones() {
   Map<String, List<Registro>> porUsuario =
         flujo(registros).collect(Collectors.groupingBy(Registro::idUsuario));

   return flujo(porUsuario.values())
         .flatMap(AnalizadorTransporte::tramosDeUsuario)
         .collect(Collectors.collectingAndThen(
         Collectors.groupingBy(TramoViaje::etiqueta, TreeMap::new,
         Collectors.collectingAndThen(
         Collectors.averagingLong(TramoViaje::segundos),
         segundos -> segundos / 60.0)),
         Collections::unmodifiableMap));
}

static Stream<TramoViaje> tramosDeUsuario(List<Registro> delUsuario) {
   List<Registro> ordenados = delUsuario.stream()
         .sorted(Comparator.comparing(Registro::timestamp))
         .toList();
   return IntStream.range(0, ordenados.size() - 1)
         .mapToObj(i -> TramoViaje.entre(ordenados.get(i), ordenados.get(i + 1)))
         .flatMap(Optional::stream);
}
```

3.6 Detección de sobrecarga (f)

Se cuentan los ingresos por ruta y por hora. Para cada ruta, estadoDeRuta toma la hora de mayor demanda, calcula la ocupación como pasajeros del pico dividido entre la capacidad horaria simulada de la ruta y marca la ruta como crítica si supera el umbral (85 % en la ejecución de evidencia).

```
public List<EstadoRuta> detectarSobrecarga(double umbralPct, ToIntFunction<String> capacidadHoraria) {
   Map<String, Map<Integer, Long>> ingresosPorRutaYHora = flujo(registros)
         .filter(ES_ENTRADA)
         .collect(Collectors.groupingBy(RUTA,
         Collectors.groupingBy(HORA_DEL_DIA, Collectors.counting())));

   return ingresosPorRutaYHora.entrySet().stream()
         .map(e -> estadoDeRuta(e.getKey(), e.getValue(), umbralPct, capacidadHoraria))

         .sorted(Comparator.comparingDouble(EstadoRuta::ocupacionPct).reversed()
         .thenComparing(EstadoRuta::ruta))
         .toList();
}

static EstadoRuta estadoDeRuta(String ruta, Map<Integer, Long> porHora,
      double umbralPct, ToIntFunction<String> capacidadHoraria) {
   Map.Entry<Integer, Long> pico = porHora.entrySet().stream()
         .max(Map.Entry.<Integer, Long>comparingByValue()
         .thenComparing(Map.Entry.comparingByKey(Comparator.reverseOrder())))
         .orElseThrow();
   double ocupacion = 100.0 * pico.getValue() / capacidadHoraria.applyAsInt(ruta);
   return new EstadoRuta(ruta, pico.getKey(), pico.getValue(), ocupacion, ocupacion > umbralPct);
}
```

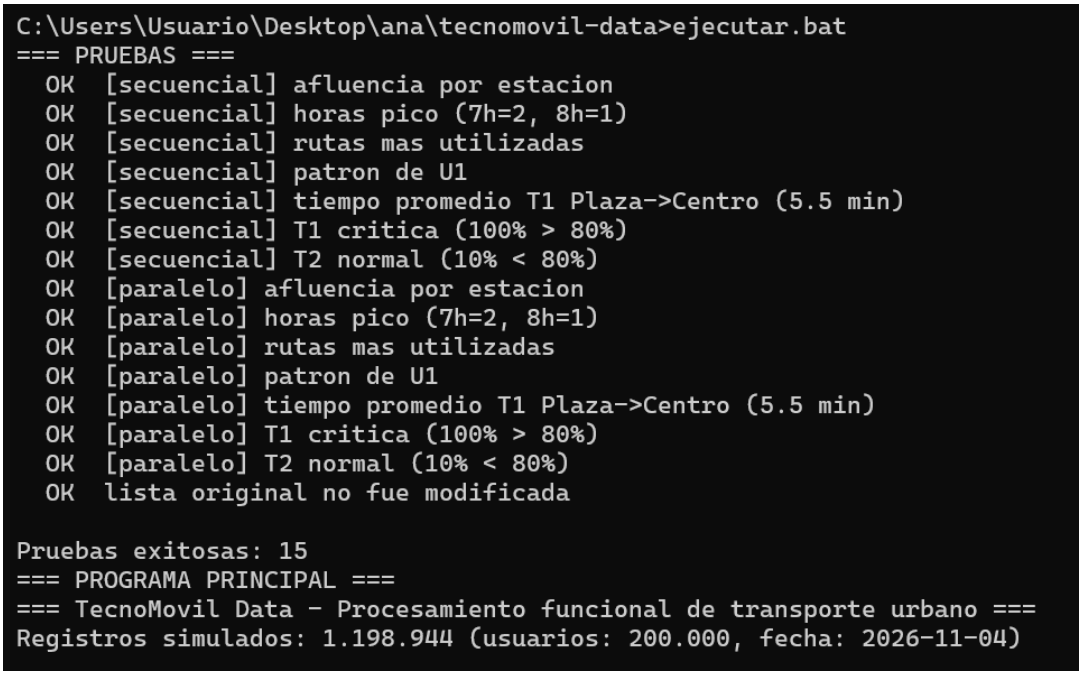
4. Pruebas y evidencias de ejecución

El proyecto se compila con javac -encoding UTF-8 -d out $(find src -name "*.java") y se ejecuta con java -cp out co.edu.iudigital.tecnomovil.Main (o con el script ejecutar.bat en Windows). Antes del programa principal se ejecutan las pruebas de PruebasFuncionales, que utilizan seis registros cuyo resultado puede calcularse a mano (por ejemplo, el tiempo promedio de la ruta T1 entre Plaza Mayor y Centro Cívico es (5 + 6) / 2 = 5,5 minutos) y verifican cada análisis en modo secuencial y paralelo, además de que la lista original no cambia. Las 15 comprobaciones resultaron exitosas (Figura 3).

Las Figuras 4 a 10 son capturas de la ejecución del programa principal: la Figura 4 muestra la generación de datos simulados y la verificación de inmutabilidad; las Figuras 5 a 9 presentan los resultados de los seis análisis sobre los 1.198.944 registros, y la Figura 10 compara la ejecución secuencial con la paralela.

Figura 3

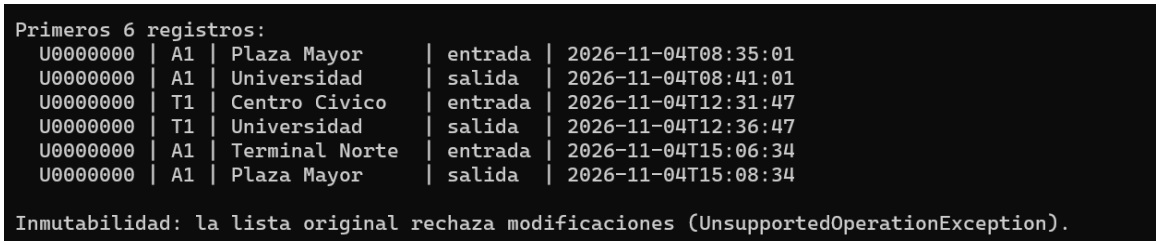
Resultado de las pruebas funcionales



Nota. Captura de la ejecución de ejecutar.bat en el símbolo del sistema de Windows.

Figura 4

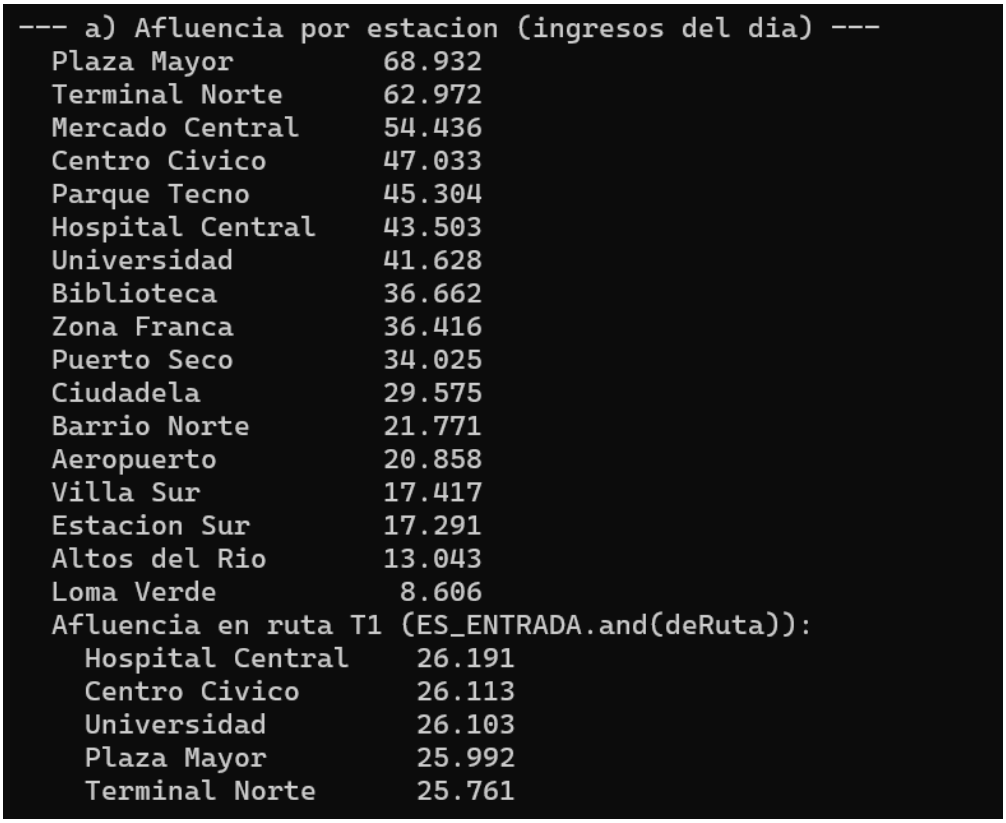
Datos simulados e inmutabilidad



Nota. Captura de la consola. La lista original rechaza modificaciones.

Figura 5

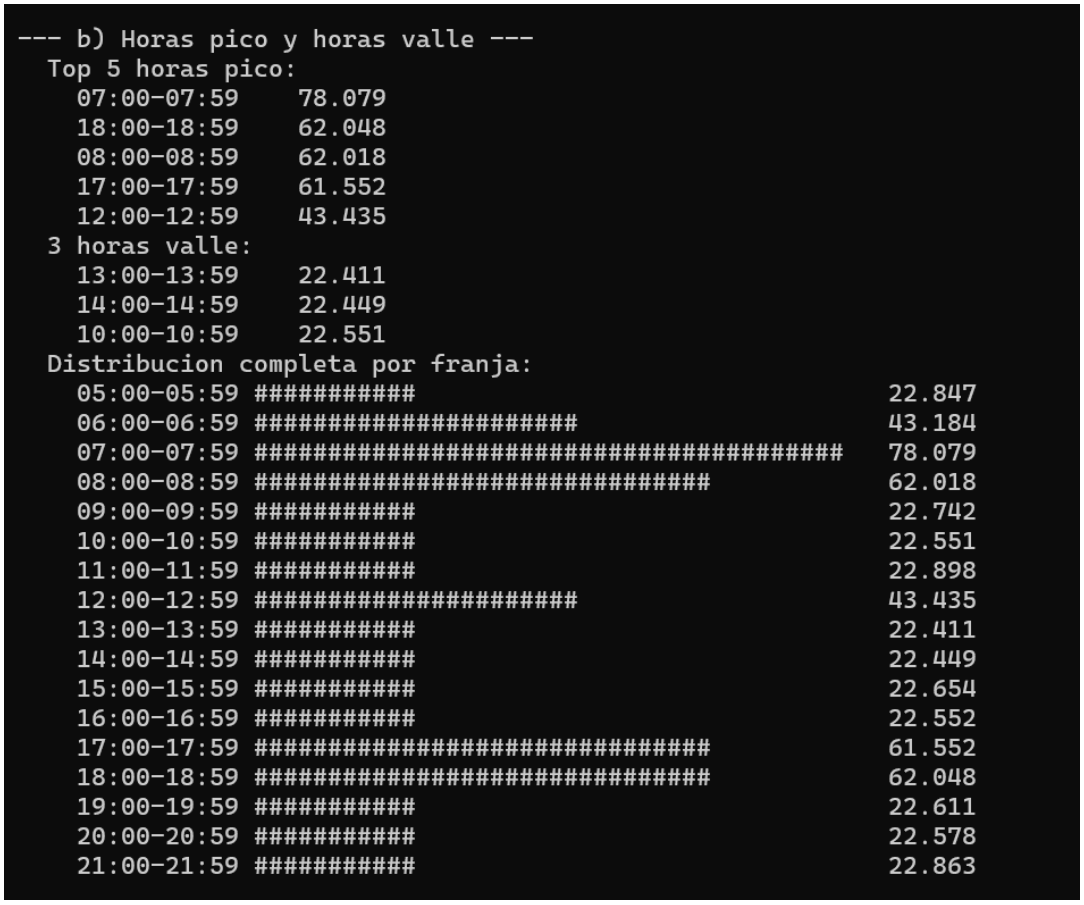
Afluencia por estación (a)



Nota. Captura de la consola. Ingresos del día por estación y, por composición de predicados, para la ruta T1.

Figura 6

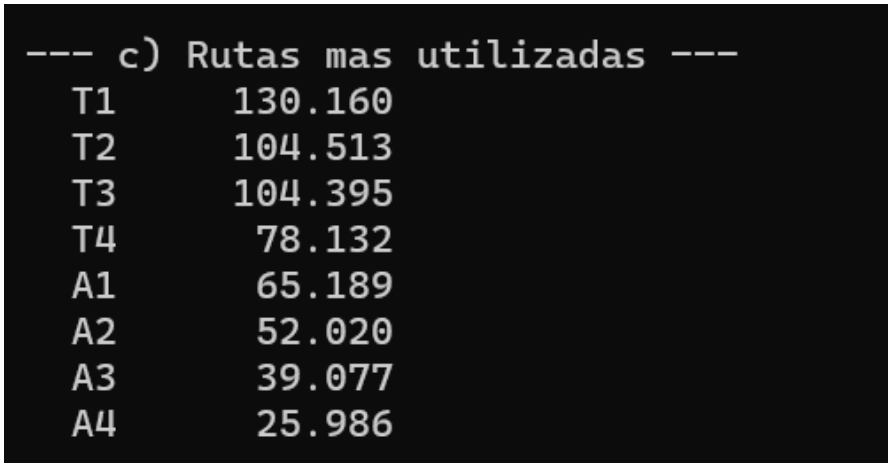
Horas pico y horas valle (b)



Nota. Captura de la consola. La hora pico es 07:00–07:59.

Figura 7

Rutas más utilizadas (c)



Nota. Captura de la consola. Ingresos por ruta, de mayor a menor.

Figura 8

Patrones de viaje por usuario (d) y tiempo promedio entre estaciones (e)

```
--- d) Patrones de viaje (primeros 5 usuarios) ---
  U0000000 -> [Plaza Mayor, Universidad, Centro Civico, Universidad, Terminal Norte, Plaza Mayor, Terminal Norte, Plaza Mayor]
  U0000001 -> [Terminal Norte, Plaza Mayor, Estacion Sur, Hospital Central, Parque Tecno, Centro Civico]
  U0000002 -> [Terminal Norte, Plaza Mayor, Parque Tecno, Centro Civico, Puerto Seco, Estacion Sur, Villa Sur, Estacion Sur]
  U0000003 -> [Plaza Mayor, Centro Civico, Centro Civico, Mercado Central, Mercado Central, Centro Civico, Terminal Norte, Plaza Mayor]
  U0000004 -> [Parque Tecno, Zona Franca, Biblioteca, Estadio, Estacion Sur, Hospital Central]

--- e) Tiempo promedio entre estaciones (min, primeros 10 tramos) ---
  A1: Barrio Norte -> Terminal Norte             4,84
  A1: Plaza Mayor -> Universidad                 4,82
  A1: Terminal Norte -> Plaza Mayor              4,80
  A2: Estacion Sur -> Hospital Central           4,81
  A2: Hospital Central -> Biblioteca             4,81
  A2: Villa Sur -> Estacion Sur                  4,84
  A3: Altos del Rio -> Puerto Seco               4,83
  A3: Mercado Central -> Centro Civico           4,82
  A3: Puerto Seco -> Mercado Central             4,84
  A4: Ciudadela -> Parque Tecno                  4,85
```

Nota. Captura de la consola (página en horizontal para conservar la legibilidad).

Figura 9

Detección de sobrecarga en rutas (f)

```
--- f) Deteccion de sobrecarga (umbral 85% de capacidad) ---
  A1   hora critica 07:00 | pico   8.512 pasajeros | ocupacion  106,4% | CRITICA
  T1   hora critica 07:00 | pico  16.953 pasajeros | ocupacion   99,7% | CRITICA
  T3   hora critica 07:00 | pico  13.674 pasajeros | ocupacion   97,7% | CRITICA
  A2   hora critica 07:00 | pico   6.741 pasajeros | ocupacion   89,9% | CRITICA
  A3   hora critica 07:00 | pico   5.092 pasajeros | ocupacion   84,9% | normal
  T4   hora critica 07:00 | pico  10.087 pasajeros | ocupacion   84,1% | normal
  T2   hora critica 07:00 | pico  13.640 pasajeros | ocupacion   80,2% | normal
  A4   hora critica 07:00 | pico   3.380 pasajeros | ocupacion   67,6% | normal
```

Nota. Captura de la consola. Umbral crítico: 85 % de la capacidad horaria simulada.

Figura 10

Comparación entre ejecución secuencial y paralela

```
--- Secuencial vs paralelo (stream() vs parallelStream()) ---
  Secuencial: 4458 ms
  Paralelo:   924 ms
  Resultados identicos: true
  Procesadores disponibles: 12
```

Nota. Captura de la consola. Los tiempos dependen del hardware.

Análisis de resultados. La estación con mayor afluencia es Plaza Mayor, la ruta más utilizada es T1 y la hora pico es 07:00–07:59, seguida de las franjas 17:00–18:59 y 08:00–08:59, coherente con los pesos de la simulación. Los tiempos promedio entre estaciones consecutivas son cercanos a 4,8 minutos, valor esperado porque los viajes simulados duran entre 2 y 9 minutos y los de hora pico son más lentos. Con el umbral de 85 %, las rutas A1, T1, T3 y A2 se marcan como críticas a las 07:00.

En la comparación entre stream() y parallelStream() (Figura 10), el equipo de ejecución, con 12 procesadores disponibles, tardó 4.458 ms en modo secuencial y 924 ms en modo paralelo, una aceleración de aproximadamente 4,8 veces, y los resultados fueron idénticos (true). Esto demuestra que los pipelines no dependen de estado compartido y que el análisis puede paralelizarse sin cambiar la lógica. La mejora concreta depende del hardware y del volumen de datos.

5. Repositorio y entregables

El proyecto se aloja en la rama main de un repositorio de GitHub e incluye el código fuente (src/main y src/test), un README.md con las instrucciones de ejecución, los scripts ejecutar.sh y ejecutar.bat, y este informe en PDF. El proyecto final se entrega además como archivo ZIP (tecnomovil-data.zip). El enlace del repositorio aparece al final de este documento.

Conclusiones

- Se cumplió el objetivo de modelar los datos con estructuras inmutables: los records, las copias defensivas y las colecciones no modificables garantizan que los registros originales no cambian durante el análisis, como lo comprueban la excepción de la Figura 4 y la prueba de la lista original.
- Los seis análisis se resolvieron con funciones puras, lambdas y pipelines de Streams sin bucles ni acumuladores mutables. El reuso de funciones de orden superior, como contarPor, redujo la duplicación y dejó el código corto, legible y fácil de mantener, atacando directamente el problema de código extenso y propenso a errores del sistema actual.
- La verificación con datos pequeños y la comparación entre stream() y parallelStream() mostraron resultados idénticos, lo que confirma que el diseño sin efectos secundarios elimina las condiciones de carrera y permite paralelizar sin cambiar la lógica; el beneficio en tiempo depende de los núcleos disponibles.
- El módulo es modular y escalable: el generador, el catálogo y el analizador están desacoplados, y la capacidad por ruta se inyecta como función, por lo que puede integrarse a TecnoMovil Data reemplazando los datos simulados por datos reales.
- La solución, su documentación y sus evidencias quedan alojadas en un repositorio de GitHub, lo que facilita el control de versiones y la trazabilidad del trabajo.

Enlaces de los repositorios

Tabla 3

Repositorio de GitHub del proyecto

Integrante	Enlace del repositorio
Ana María Osorio Tabarez	https://github.com/anaosoriot/tecnomovil-data

Nota. Repositorio en la rama main.

Referencias

Bloch, J. (2018). Effective Java (3.ª ed.). Addison-Wesley.

Institución Universitaria Digital de Antioquia. (2026). Unidad 2: Caso de estudio. Tecnología en Desarrollo de Software [Material de curso]. IU Digital.

Oracle. (s. f.-a). Record classes. Java Language Updates, Java SE 21. https://docs.oracle.com/en/java/javase/21/language/records.html

Oracle. (s. f.-b). Lambda expressions. The Java Tutorials. https://docs.oracle.com/javase/tutorial/java/javaOO/lambdaexpressions.html

Oracle. (s. f.-c). Package java.util.stream. Java SE 21 & JDK 21 API. https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/stream/package-summary.html

Anexo A. Código fuente en Java

A continuación se presenta el código fuente completo del proyecto (sin frameworks externos).

src/main/java/co/edu/iudigital/tecnomovil/Main.java

```
package co.edu.iudigital.tecnomovil;

import co.edu.iudigital.tecnomovil.datos.Catalogo;
import co.edu.iudigital.tecnomovil.datos.GeneradorDatos;
import co.edu.iudigital.tecnomovil.funcional.AnalizadorTransporte;
import co.edu.iudigital.tecnomovil.modelo.EstadoRuta;
import co.edu.iudigital.tecnomovil.modelo.Registro;

import java.time.LocalDate;
import java.util.List;
import java.util.Map;

/**
 * Punto de entrada. Es la "capa imperativa" del programa: aquí (y solo aquí) ocurren
 * los efectos secundarios de entrada/salida (imprimir en consola). Todo el cálculo
 * está en {@link AnalizadorTransporte}, que es puro.
 */
public final class Main {

    private static final int USUARIOS = 200_000;
    private static final long SEMILLA = 2026L;
    private static final LocalDate FECHA = LocalDate.of(2026, 11, 4);
    private static final double UMBRAL_CRITICO_PCT = 85.0;

    public static void main(String[] args) {
        System.out.println("=== TecnoMovil Data - Procesamiento funcional de transporte urbano ===");

        List<Registro> registros = GeneradorDatos.generar(USUARIOS, SEMILLA, FECHA);
        System.out.printf("Registros simulados: %,d (usuarios: %,d, fecha: %s)%n%n",
                registros.size(), USUARIOS, FECHA);

        System.out.println("Primeros 6 registros:");
        registros.stream().limit(6).forEach(r -> System.out.printf(
                " %s | %s | %-15s | %-7s | %s%n",
                r.idUsuario(), r.ruta(), r.estacion(), r.accion().texto(), r.timestamp()));

        verificarInmutabilidad(registros);

        AnalizadorTransporte secuencial = new AnalizadorTransporte(registros, false);

        System.out.println("\n--- a) Afluencia por estacion (ingresos del dia) ---");
        secuencial.afluenciaPorEstacion()
                .forEach((estacion, total) -> System.out.printf(" %-16s %,9d%n", estacion, total));
        System.out.println(" Afluencia en ruta T1 (ES_ENTRADA.and(deRuta)):");
        secuencial.afluenciaPorEstacionEnRuta("T1")
                .forEach((estacion, total) -> System.out.printf("    %-16s %,9d%n", estacion, total));

        System.out.println("\n--- b) Horas pico y horas valle ---");
        System.out.println(" Top 5 horas pico:");
        imprimirHoras(secuencial.horasPico(5));
        System.out.println(" 3 horas valle:");
        imprimirHoras(secuencial.horasValle(3));
        System.out.println(" Distribucion completa por franja:");
        Map<String, Long> franjas = secuencial.ingresosPorFranja();
        long maximo = franjas.values().stream().mapToLong(Long::longValue).max().orElse(1);
```

```
    franjas.forEach((franja, total) -> System.out.printf("    %s %-40s %,8d%n",
            franja, "#".repeat((int) (40 * total / maximo)), total));

    System.out.println("\n--- c) Rutas mas utilizadas ---");
    secuencial.rutasMasUtilizadas(8)
            .forEach((ruta, total) -> System.out.printf(" %-4s %,9d%n", ruta, total));

    System.out.println("\n--- d) Patrones de viaje (primeros 5 usuarios) ---");
    secuencial.patronesDeViaje().entrySet().stream().limit(5)

            .forEach(e -> System.out.printf("   %s -> %s%n", e.getKey(), e.getValue()));

    System.out.println("\n--- e) Tiempo promedio entre estaciones (min, primeros 10 tramos) ---");
    secuencial.tiempoPromedioEntreEstaciones().entrySet().stream().limit(10)
            .forEach(e -> System.out.printf(" %-45s %5.2f%n", e.getKey(), e.getValue()));

    System.out.printf("%n--- f) Deteccion de sobrecarga (umbral %.0f%% de capacidad) ---%n",
            UMBRAL_CRITICO_PCT);
    secuencial.detectarSobrecarga(UMBRAL_CRITICO_PCT, Catalogo.CAPACIDAD_POR_RUTA::apply)
            .forEach(Main::imprimirEstadoRuta);

    compararSecuencialYParalelo(registros);
}

private static void imprimirHoras(Map<Integer, Long> horas) {
    horas.forEach((hora, total) ->
            System.out.printf("    %02d:00-%02d:59 %,9d%n", hora, hora, total));
}

private static void imprimirEstadoRuta(EstadoRuta e) {
    System.out.printf(" %-4s hora critica %02d:00 | pico %,7d pasajeros | ocupacion %6.1f%% | %s%n",
            e.ruta(), e.horaCritica(), e.pasajerosPico(), e.ocupacionPct(),
            e.critica() ? "CRITICA" : "normal");
}

/** Demuestra que la colección original es inmutable. */
private static void verificarInmutabilidad(List<Registro> registros) {
    try {
        registros.add(registros.get(0));
        System.out.println("\n[ERROR] la lista original se pudo modificar");
    } catch (UnsupportedOperationException e) {
        System.out.println("\nInmutabilidad: la lista original rechaza modificaciones "
                + "(UnsupportedOperationException).");
    }
}

/** Ejecuta todo el análisis en modo secuencial y paralelo y compara resultados y tiempos. */
private static void compararSecuencialYParalelo(List<Registro> registros) {
    System.out.println("\n--- Secuencial vs paralelo (stream() vs parallelStream()) ---");
    var secuencial = new AnalizadorTransporte(registros, false);
    var paralelo = new AnalizadorTransporte(registros, true);

    ejecutarTodo(secuencial); // calentamiento de la JVM
    ejecutarTodo(paralelo);

    long t0 = System.nanoTime();
    var resSecuencial = ejecutarTodo(secuencial);
    long t1 = System.nanoTime();
    var resParalelo = ejecutarTodo(paralelo);
    long t2 = System.nanoTime();

    System.out.printf(" Secuencial: %d ms%n", (t1 - t0) / 1_000_000);
```

```
        System.out.printf(" Paralelo:   %d ms%n", (t2 - t1) / 1_000_000);
        System.out.println(" Resultados identicos: " + resSecuencial.equals(resParalelo));
        System.out.printf(" Procesadores disponibles: %d%n", Runtime.getRuntime().availableProcessors());
    }

    private static List<Object> ejecutarTodo(AnalizadorTransporte a) {
        return List.of(
                a.afluenciaPorEstacion(),
                a.distribucionPorHora(),
                a.rutasMasUtilizadas(8),
                a.patronesDeViaje(),
                a.tiempoPromedioEntreEstaciones(),
                a.detectarSobrecarga(UMBRAL_CRITICO_PCT, Catalogo.CAPACIDAD_POR_RUTA::apply));
    }

    private Main() {
    }
}
```

src/main/java/co/edu/iudigital/tecnomovil/modelo/Accion.java

```
package co.edu.iudigital.tecnomovil.modelo;

/**

 * Acciones posibles de un registro de validación.
 * Se modela como enum (tipo cerrado e inmutable) en lugar de un String libre.
 */
public enum Accion {
    ENTRADA("entrada"),
           SALIDA("salida");

    private final String texto;

    Accion(String texto) {
        this.texto = texto;
    }

    public String texto() {
        return texto;
    }

    /** Función pura: convierte "entrada"/"salida" en la constante correspondiente. */
    public static Accion desdeTexto(String texto) {
        return switch (texto.toLowerCase()) {
            case "entrada" -> ENTRADA;
            case "salida" -> SALIDA;
            default -> throw new IllegalArgumentException("Acción desconocida: " + texto);
        };
    }
}
```

src/main/java/co/edu/iudigital/tecnomovil/modelo/Registro.java

```
package co.edu.iudigital.tecnomovil.modelo;

import java.time.LocalDateTime;
import java.util.Objects;

/**
```

```
 * Registro de validación (sección 3.1 del caso de estudio).
 * Al ser un {@code record} es inmutable: no tiene setters y todos sus campos son finales.
 */
public record Registro(String idUsuario,
        String ruta,
        String estacion,
        Accion accion,
        LocalDateTime timestamp) {

    public Registro {
        Objects.requireNonNull(idUsuario, "idUsuario");
        Objects.requireNonNull(ruta, "ruta");
        Objects.requireNonNull(estacion, "estacion");
        Objects.requireNonNull(accion, "accion");
        Objects.requireNonNull(timestamp, "timestamp");
    }
}
```

src/main/java/co/edu/iudigital/tecnomovil/modelo/Ruta.java

```
package co.edu.iudigital.tecnomovil.modelo;

import java.util.List;

/**
 * Definición (inmutable) de una ruta del sistema.
 *
 * @param id         identificador de la ruta (T = troncal, A = alimentadora)
 * @param estaciones estaciones en orden de recorrido
 * @param peso       popularidad relativa usada por el simulador de datos
 * @param capacidadHoraria pasajeros por hora que la ruta puede mover (valor simulado)
 */
public record Ruta(String id, List<String> estaciones, int peso, int capacidadHoraria) {

    public Ruta {
        estaciones = List.copyOf(estaciones); // copia defensiva inmutable
    }
}
```

src/main/java/co/edu/iudigital/tecnomovil/modelo/TramoViaje.java

```

package co.edu.iudigital.tecnomovil.modelo;

import java.time.Duration;
import java.util.Optional;

/**
 * Tramo recorrido por un usuario entre dos estaciones consecutivas de sus registros.
 */
public record TramoViaje(String ruta, String origen, String destino, long segundos) {

    /**
     * Función pura: un par de registros consecutivos forma un tramo válido solo si
     * pertenecen a la misma ruta, el primero es ENTRADA, el segundo es SALIDA y
     * las estaciones son distintas.
     */
    public static Optional<TramoViaje> entre(Registro a, Registro b) {
        boolean valido = a.ruta().equals(b.ruta())
                && a.accion() == Accion.ENTRADA
```

```
            && b.accion() == Accion.SALIDA
            && !a.estacion().equals(b.estacion());
        return valido
            ? Optional.of(new TramoViaje(a.ruta(), a.estacion(), b.estacion(),
            Duration.between(a.timestamp(), b.timestamp()).getSeconds()))
            : Optional.empty();
    }

    /** Etiqueta legible usada como llave de agrupación. */
    public String etiqueta() {
        return ruta + ": " + origen + " -> " + destino;
    }
}
```

src/main/java/co/edu/iudigital/tecnomovil/modelo/EstadoRuta.java

```
package co.edu.iudigital.tecnomovil.modelo;

/**
 * Resultado del análisis de sobrecarga para una ruta.
 *
 * @param ruta     identificador de la ruta
 * @param horaCritica
hora del día con más ingresos en la ruta
 * @param pasajerosPico ingresos registrados en esa hora
 * @param ocupacionPct
pasajerosPico / capacidadHoraria * 100
 * @param critica  true si la ocupación supera el umbral
 */
public record EstadoRuta(String ruta, int horaCritica, long pasajerosPico,
        double ocupacionPct, boolean critica) {
}
```

src/main/java/co/edu/iudigital/tecnomovil/datos/Catalogo.java

```
package co.edu.iudigital.tecnomovil.datos;

import co.edu.iudigital.tecnomovil.modelo.Ruta;

import java.util.List;
import java.util.Map;
import java.util.function.Function;
import java.util.stream.Collectors;

/**
 * Catálogo inmutable de rutas y estaciones de TecnoValle (datos simulados).
 * Se usa List.of para que el orden de iteración sea estable entre ejecuciones.
 */
public final class Catalogo {

    public static final List<Ruta> RUTAS = List.of(
            new Ruta("T1", List.of("Terminal Norte", "Plaza Mayor", "Centro Civico",
            "Universidad", "Hospital Central", "Estacion Sur"), 10, 17000),
            new Ruta("T2", List.of("Aeropuerto", "Zona Franca", "Plaza Mayor",
            "Mercado Central", "Biblioteca", "Estadio"), 8, 17000),
            new Ruta("T3", List.of("Ciudadela", "Parque Tecno", "Centro Civico",
            "Mercado Central", "Puerto Seco", "Estacion Sur"), 8, 14000),
            new Ruta("T4", List.of("Terminal Norte", "Universidad", "Biblioteca",
            "Parque Tecno", "Zona Franca", "Ciudadela"), 6, 12000),
```

```
            new Ruta("A1", List.of("Barrio Norte", "Terminal Norte", "Plaza Mayor",
            "Universidad"), 5, 8000),
            new Ruta("A2", List.of("Villa Sur", "Estacion Sur", "Hospital Central",
            "Biblioteca"), 4, 7500),
            new Ruta("A3", List.of("Altos del Rio", "Puerto Seco", "Mercado Central",
            "Centro Civico"), 3, 6000),
            new Ruta("A4", List.of("Loma Verde", "Ciudadela", "Parque Tecno",
            "Estadio"), 2, 5000));

    /** Capacidad horaria por ruta, derivada del catálogo (función de consulta, sin estado). */
    private static final Map<String, Integer> CAPACIDADES = RUTAS.stream()
            .collect(Collectors.toUnmodifiableMap(Ruta::id, Ruta::capacidadHoraria));

    public static final Function<String, Integer> CAPACIDAD_POR_RUTA = CAPACIDADES::get;

    private Catalogo() {
    }
}
```

src/main/java/co/edu/iudigital/tecnomovil/datos/GeneradorDatos.java

```
package co.edu.iudigital.tecnomovil.datos;

import co.edu.iudigital.tecnomovil.modelo.Accion;
import co.edu.iudigital.tecnomovil.modelo.Registro;
import co.edu.iudigital.tecnomovil.modelo.Ruta;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.Collections;
import java.util.List;
import java.util.SplittableRandom;
import java.util.stream.Collectors;
import java.util.stream.IntStream;
import java.util.stream.Stream;

/**
 * Simulador de registros de validadores.
 *
 * Es determinista: dadas las mismas entradas (cantidad, semilla, fecha) produce
 * siempre la misma lista. Cada viaje se calcula como función pura de
 * (semilla, usuario, número de viaje), por lo que la generación puede ser paralela.
 */
public final class GeneradorDatos {

    /** Horas de inicio de viaje; las repeticiones modelan las horas pico (7-8 y 17-18). */
    private static final List<Integer> HORAS_PONDERADAS = List.of(
            5, 6, 6, 7, 7, 7, 7, 8, 8, 8, 9, 10, 11, 12, 12,
            13, 14, 15, 16, 17, 17, 17, 18, 18, 18, 19, 20, 21);

    /** Rutas repetidas según su peso: un índice al azar equivale a un sorteo ponderado. */
    private static final List<Ruta> RUTAS_PONDERADAS = Catalogo.RUTAS.stream()
            .flatMap(r -> Collections.nCopies(r.peso(), r).stream())
            .collect(Collectors.toUnmodifiableList());

    private GeneradorDatos() {
    }

    public static List<Registro> generar(int cantidadUsuarios, long semilla, LocalDate fecha) {
        return IntStream.range(0, cantidadUsuarios)
                .parallel()
```

```
            .boxed()
            .flatMap(u -> viajesDeUsuario(u, semilla, fecha))
            .toList(); // lista inmutable; el flujo ordenado conserva el orden de los usuarios
    }

    private static Stream<Registro> viajesDeUsuario(int usuario, long semilla, LocalDate fecha) {
        String id = String.format("U%07d", usuario);
        SplittableRandom rnd = new SplittableRandom(mezclar(semilla, usuario, -1));
        int cantidadViajes = 2 + rnd.nextInt(3); // 2..4 viajes por usuario

        // Horas distintas para que los viajes de un mismo usuario no se solapen.
        List<Integer> horas = rnd.ints(8, 0, HORAS_PONDERADAS.size())
                .boxed()
                .map(HORAS_PONDERADAS::get)
                .distinct()

                .limit(cantidadViajes)
                .toList();

        return IntStream.range(0, horas.size())
                .boxed()
                .flatMap(v -> viaje(id, horas.get(v),
                new SplittableRandom(mezclar(semilla, usuario, v)), fecha));
    }

    private static Stream<Registro> viaje(String idUsuario, int hora, SplittableRandom rnd, LocalDate fecha) {
        LocalDateTime inicio = fecha.atTime(hora, rnd.nextInt(50), rnd.nextInt(60));

        Ruta ruta = RUTAS_PONDERADAS.get(rnd.nextInt(RUTAS_PONDERADAS.size()));
        int origen = rnd.nextInt(ruta.estaciones().size() - 1);
        int destino = origen + 1; // estaciones consecutivas

        boolean horaPico = hora == 7 || hora == 8 || hora == 17 || hora == 18;
        int minutosViaje = 2 + rnd.nextInt(4) + (horaPico ? 3 : 0);

        return Stream.of(
                new Registro(idUsuario, ruta.id(), ruta.estaciones().get(origen),
                Accion.ENTRADA, inicio),
                new Registro(idUsuario, ruta.id(), ruta.estaciones().get(destino),
                Accion.SALIDA, inicio.plusMinutes(minutosViaje)));
    }

    /** Mezcla de bits (tipo splitmix) para obtener semillas independientes por viaje. */
    private static long mezclar(long semilla, int usuario, int viaje) {
        long z = semilla + usuario * 0x9E3779B97F4A7C15L + (viaje + 1) * 0xBF58476D1CE4E5B9L;
        z = (z ^ (z >>> 30)) * 0xBF58476D1CE4E5B9L;
        z = (z ^ (z >>> 27)) * 0x94D049BB133111EBL;
        return z ^ (z >>> 31);
    }
}
```

src/main/java/co/edu/iudigital/tecnomovil/funcional/Funciones.java

```
package co.edu.iudigital.tecnomovil.funcional;

import co.edu.iudigital.tecnomovil.modelo.Accion;
import co.edu.iudigital.tecnomovil.modelo.Registro;

import java.util.function.Function;
import java.util.function.Predicate;
```

```
/**
 * Biblioteca de funciones puras reutilizables (lambdas con nombre).
 * Ninguna modifica estado externo: el resultado depende solo de los argumentos.
 */
public final class Funciones {

    // ---- Predicados ----
    public static final Predicate<Registro> ES_ENTRADA = r -> r.accion() == Accion.ENTRADA;
    public static final Predicate<Registro> ES_SALIDA = r -> r.accion() == Accion.SALIDA;

    /** Fábrica de predicados: función de orden superior que RETORNA una función. */
    public static Predicate<Registro> deRuta(String ruta) {
        return r -> r.ruta().equals(ruta);
    }

    // ---- Clasificadores (funciones de extracción) ----
    public static final Function<Registro, String> ESTACION = Registro::estacion;
    public static final Function<Registro, String> RUTA = Registro::ruta;
    public static final Function<Registro, Integer> HORA_DEL_DIA = r -> r.timestamp().getHour();

    // ---- Composición de funciones (andThen) ----
    public static final Function<Registro, String> FRANJA_HORARIA =
            HORA_DEL_DIA.andThen(h -> String.format("%02d:00-%02d:59", h, h));

    private Funciones() {
    }
}
```

src/main/java/co/edu/iudigital/tecnomovil/funcional/AnalizadorTransporte.java

```
package co.edu.iudigital.tecnomovil.funcional;

import static co.edu.iudigital.tecnomovil.funcional.Funciones.ES_ENTRADA;
import static co.edu.iudigital.tecnomovil.funcional.Funciones.FRANJA_HORARIA;
import static co.edu.iudigital.tecnomovil.funcional.Funciones.ESTACION;
import static co.edu.iudigital.tecnomovil.funcional.Funciones.HORA_DEL_DIA;
import static co.edu.iudigital.tecnomovil.funcional.Funciones.RUTA;
import static co.edu.iudigital.tecnomovil.funcional.Funciones.deRuta;

import co.edu.iudigital.tecnomovil.modelo.EstadoRuta;
import co.edu.iudigital.tecnomovil.modelo.Registro;
import co.edu.iudigital.tecnomovil.modelo.TramoViaje;

import java.util.Collection;
import java.util.Collections;
import java.util.Comparator;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.TreeMap;
import java.util.function.Function;
import java.util.function.Predicate;
import java.util.function.ToIntFunction;
import java.util.stream.Collectors;
import java.util.stream.IntStream;
import java.util.stream.Stream;

/**
 * Módulo funcional de procesamiento de datos diarios de TecnoMovil Data.
```

```
 *
 * Diseño:
 * - La instancia es inmutable: guarda una copia inmutable de los registros (List.copyOf).
 * - Todos los métodos son puros: no modifican la lista, no usan variables compartidas
 *   y devuelven colecciones no modificables.
 * - Si {@code paralelo} es true, cada pipeline usa parallelStream(); como no hay estado
 *   compartido mutable, el resultado es idéntico al secuencial.
 */
public final class AnalizadorTransporte {

  private final List<Registro> registros;
  private final boolean paralelo;

  public AnalizadorTransporte(List<Registro> registros, boolean paralelo) {
     this.registros = List.copyOf(registros); // copia defensiva inmutable
     this.paralelo = paralelo;
  }

  // ========================================================================
  // a) Afluencia por estación
  // ========================================================================
  public Map<String, Long> afluenciaPorEstacion() {
     return ordenarDescendente(contarPor(ES_ENTRADA, ESTACION), Integer.MAX_VALUE);
  }

  /** Variante que demuestra composición de predicados: ES_ENTRADA.and(deRuta(...)). */
  public Map<String, Long> afluenciaPorEstacionEnRuta(String ruta) {
     return ordenarDescendente(contarPor(ES_ENTRADA.and(deRuta(ruta)), ESTACION), Integer.MAX_VALUE);
  }

  // ========================================================================
  // b) Horas pico y horas valle
  // ========================================================================
  /** Ingresos por hora del día, ordenados por hora (0-23). */
  public Map<Integer, Long> distribucionPorHora() {
     return Collections.unmodifiableMap(new TreeMap<>(contarPor(ES_ENTRADA, HORA_DEL_DIA)));
  }

  /** Ingresos por franja horaria ("07:00-07:59"), en orden cronológico. Usa composición andThen. */
  public Map<String, Long> ingresosPorFranja() {
     return Collections.unmodifiableMap(new TreeMap<>(contarPor(ES_ENTRADA, FRANJA_HORARIA)));
  }

  public Map<Integer, Long> horasPico(int n) {
     return ordenarDescendente(contarPor(ES_ENTRADA, HORA_DEL_DIA), n);
  }

  public Map<Integer, Long> horasValle(int n) {
     Map<Integer, Long> porHora = contarPor(ES_ENTRADA, HORA_DEL_DIA);
     Comparator<Map.Entry<Integer, Long>> ascendente =
           Map.Entry.<Integer, Long>comparingByValue().thenComparing(Map.Entry.comparingByKey());
     return ordenar(porHora, ascendente, n);
  }

  // ========================================================================
  // c) Rutas más utilizadas
  // ========================================================================
  public Map<String, Long> rutasMasUtilizadas(int n) {
     return ordenarDescendente(contarPor(ES_ENTRADA, RUTA), n);
  }

  // ========================================================================
```

```
// d) Patrones de viaje por usuario
// ========================================================================
/**
 * Estaciones visitadas por cada usuario, en el orden de sus registros
 * (ordenados por timestamp; el orden es estable ante empates).
 */
public Map<String, List<String>> patronesDeViaje() {
    return flujo(registros)
            .sorted(Comparator.comparing(Registro::timestamp))
            .collect(Collectors.collectingAndThen(
            Collectors.groupingBy(Registro::idUsuario, TreeMap::new,
            Collectors.mapping(ESTACION,
            Collectors.collectingAndThen(Collectors.toList(), List::copyOf))),
            Collections::unmodifiableMap));
}

// ========================================================================
// e) Tiempo promedio (minutos) entre estaciones consecutivas
// ========================================================================
public Map<String, Double> tiempoPromedioEntreEstaciones() {
    Map<String, List<Registro>> porUsuario =
            flujo(registros).collect(Collectors.groupingBy(Registro::idUsuario));

    return flujo(porUsuario.values())
            .flatMap(AnalizadorTransporte::tramosDeUsuario)
            .collect(Collectors.collectingAndThen(
            Collectors.groupingBy(TramoViaje::etiqueta, TreeMap::new,
            Collectors.collectingAndThen(
            Collectors.averagingLong(TramoViaje::segundos),
            segundos -> segundos / 60.0)),
            Collections::unmodifiableMap));
}

/** Función pura: pares consecutivos de registros de un usuario -> tramos válidos. */
static Stream<TramoViaje> tramosDeUsuario(List<Registro> delUsuario) {
    List<Registro> ordenados = delUsuario.stream()
            .sorted(Comparator.comparing(Registro::timestamp))
            .toList();
    return IntStream.range(0, ordenados.size() - 1)
            .mapToObj(i -> TramoViaje.entre(ordenados.get(i), ordenados.get(i + 1)))
            .flatMap(Optional::stream);
}

// ========================================================================
// f) Detección de sobrecarga en rutas
// ========================================================================
/**
 * Marca como crítica la ruta cuya hora de mayor demanda supera el umbral de ocupación.
 *
 * @param umbralPct porcentaje (0-100) a partir del cual la ruta es crítica
 * @param capacidadHoraria función que entrega la capacidad simulada de cada ruta
 *                  (se inyecta como parámetro: función de orden superior)
 */
public List<EstadoRuta> detectarSobrecarga(double umbralPct, ToIntFunction<String> capacidadHoraria) {
    Map<String, Map<Integer, Long>> ingresosPorRutaYHora = flujo(registros)
            .filter(ES_ENTRADA)
            .collect(Collectors.groupingBy(RUTA,
            Collectors.groupingBy(HORA_DEL_DIA, Collectors.counting())));

    return ingresosPorRutaYHora.entrySet().stream()
            .map(e -> estadoDeRuta(e.getKey(), e.getValue(), umbralPct, capacidadHoraria))
```

```
            .sorted(Comparator.comparingDouble(EstadoRuta::ocupacionPct).reversed()
            .thenComparing(EstadoRuta::ruta))
            .toList();
    }

    /** Función pura: calcula el estado de una ruta a partir de sus ingresos por hora. */
    static EstadoRuta estadoDeRuta(String ruta, Map<Integer, Long> porHora,
            double umbralPct, ToIntFunction<String> capacidadHoraria) {
        Map.Entry<Integer, Long> pico = porHora.entrySet().stream()
                .max(Map.Entry.<Integer, Long>comparingByValue()
                .thenComparing(Map.Entry.comparingByKey(Comparator.reverseOrder())))
                .orElseThrow();
        double ocupacion = 100.0 * pico.getValue() / capacidadHoraria.applyAsInt(ruta);
        return new EstadoRuta(ruta, pico.getKey(), pico.getValue(), ocupacion, ocupacion > umbralPct);
    }

    // ========================================================================
    // Utilidades genéricas (funciones de orden superior)
    // ========================================================================

    /** Cuenta registros que cumplen {@code filtro}, agrupados por {@code clasificador}. */
    private <K> Map<K, Long> contarPor(Predicate<Registro> filtro, Function<Registro, K> clasificador) {
        return flujo(registros)
                .filter(filtro)
                .collect(Collectors.groupingBy(clasificador, Collectors.counting()));
    }

    private <T> Stream<T> flujo(Collection<T> coleccion) {
        return paralelo ? coleccion.parallelStream() : coleccion.stream();
    }

    private static <K extends Comparable<K>, V extends Comparable<V>> Map<K, V> ordenarDescendente(
            Map<K, V> mapa, int limite) {
        Comparator<Map.Entry<K, V>> descendente =
                Map.Entry.<K, V>comparingByValue().reversed().thenComparing(Map.Entry.comparingByKey());
        return ordenar(mapa, descendente, limite);
    }

    private static <K, V> Map<K, V> ordenar(Map<K, V> mapa, Comparator<Map.Entry<K, V>> criterio, int limite) {
        return mapa.entrySet().stream()
                .sorted(criterio)
                .limit(limite)
                .collect(Collectors.collectingAndThen(
                Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue,
                (a, b) -> a, LinkedHashMap::new),
                Collections::unmodifiableMap));
    }
}
```

src/test/java/co/edu/iudigital/tecnomovil/PruebasFuncionales.java

```
package co.edu.iudigital.tecnomovil;

import co.edu.iudigital.tecnomovil.funcional.AnalizadorTransporte;
import co.edu.iudigital.tecnomovil.modelo.Accion;
import co.edu.iudigital.tecnomovil.modelo.EstadoRuta;
import co.edu.iudigital.tecnomovil.modelo.Registro;

import java.time.LocalDateTime;
import java.util.List;
import java.util.Map;
```

```
/**
 * Pruebas manuales (sin frameworks externos) sobre un conjunto pequeño y verificable a mano.
 * Ejecutar con: java -cp out co.edu.iudigital.tecnomovil.PruebasFuncionales
 */
public final class PruebasFuncionales {

  private static int exitosas = 0;

  public static void main(String[] args) {
    List<Registro> datos = List.of(
        reg("U1", "T1", "Plaza Mayor", Accion.ENTRADA, 7, 0),
        reg("U1", "T1", "Centro Civico", Accion.SALIDA, 7, 5),
        reg("U2", "T1", "Plaza Mayor", Accion.ENTRADA, 7, 10),
        reg("U2", "T1", "Centro Civico", Accion.SALIDA, 7, 16),

        reg("U3", "T2", "Plaza Mayor", Accion.ENTRADA, 8, 0),
        reg("U3", "T2", "Mercado Central", Accion.SALIDA, 8, 4));
    List<Registro> copiaOriginal = List.copyOf(datos);

    for (boolean paralelo : new boolean[]{false, true}) {
      String modo = paralelo ? "[paralelo] " : "[secuencial] ";
      AnalizadorTransporte a = new AnalizadorTransporte(datos, paralelo);

      comprobar(modo + "afluencia por estacion",
          Map.of("Plaza Mayor", 3L), a.afluenciaPorEstacion());
      comprobar(modo + "horas pico (7h=2, 8h=1)",
          Map.of(7, 2L, 8, 1L), a.horasPico(2));
      comprobar(modo + "rutas mas utilizadas",
          Map.of("T1", 2L, "T2", 1L), a.rutasMasUtilizadas(5));
      comprobar(modo + "patron de U1",
          List.of("Plaza Mayor", "Centro Civico"), a.patronesDeViaje().get("U1"));
      comprobar(modo + "tiempo promedio T1 Plaza->Centro (5.5 min)",
          5.5, a.tiempoPromedioEntreEstaciones().get("T1: Plaza Mayor -> Centro Civico"));

      List<EstadoRuta> estados = a.detectarSobrecarga(80.0, ruta -> ruta.equals("T1") ? 2 : 10);
      comprobar(modo + "T1 critica (100% > 80%)", true, estadoDe(estados, "T1").critica());
      comprobar(modo + "T2 normal (10% < 80%)", false, estadoDe(estados, "T2").critica());
    }

    comprobar("lista original no fue modificada", copiaOriginal, datos);
    System.out.printf("%nPruebas exitosas: %d%n", exitosas);
  }

  private static EstadoRuta estadoDe(List<EstadoRuta> estados, String ruta) {
    return estados.stream().filter(e -> e.ruta().equals(ruta)).findFirst().orElseThrow();
  }

  private static Registro reg(String u, String ruta, String est, Accion acc, int h, int m) {
    return new Registro(u, ruta, est, acc, LocalDateTime.of(2026, 11, 4, h, m));
  }

  private static void comprobar(String nombre, Object esperado, Object obtenido) {
    if (!esperado.equals(obtenido)) {
      throw new AssertionError(nombre + " -> esperado " + esperado + " pero fue " + obtenido);
    }
    exitosas++;
    System.out.println(" OK " + nombre);
  }

  private PruebasFuncionales() {
  }
```

```
}
```